

INSTITUTO TECNOLÓGICO UNIVERSITARIO

Proyecto Integrador — EGI

ECOSISTEMA DE INVENTARIO
SEGURO

Documentación Técnica Profesional

Institución	Instituto Tecnológico Universitario (ITU)
Materia	Proyecto Integrador / EGI
Versión	1.0 — Release Candidato
Año Académico	2025
Clasificación	Uso Interno Académico

CONFIDENCIAL — DOCUMENTACIÓN ACADÉMICA

Tabla de Contenidos

Tabla de Contenidos	2
1. Introducción	3
1.1 Descripción General del Proyecto	3
1.2 Objetivo Principal	3
1.3 Problemática que Resuelve	3
1.4 Contexto del Sistema	3
2. Decisiones de Diseño	4
2.1 Arquitectura Seleccionada: Microservicios Contenerizados	4
2.2 Tecnologías Elegidas y Justificación	4
2.3 Justificación de la Base de Datos Dual	5
2.3.1 SQL Server — Datos Estructurados (ubicacion-db)	5
2.3.2 MongoDB — Datos Documentales (inventario-db)	5
2.4 Patrones de Diseño Aplicados	6
2.5 Decisiones de Seguridad	6
2.6 Ventajas y Desventajas del Diseño	6
3. Roles del Proyecto	7
3.1 Estructura del Equipo	7
3.2 Metodología de Trabajo	7
3.3 Flujo de Trabajo Git	8
4. Requisitos Funcionales	8
5. Requisitos No Funcionales	9
6. Casos de Uso	10
6.1 Actores del Sistema	10
6.2 Casos de Uso Detallados	11
CU-01: Autenticación de Usuario	11
CU-02: Consulta de Inventario (Listado)	11
CU-03: Consulta de Detalle de Equipo	12
CU-04: Gestión CRUD de Hardware (MongoDB)	12
7. Arquitectura del Proyecto	13
7.1 Visión General	13
7.2 Capas de la Arquitectura	13
7.2.1 Capa de Presentación (Frontend)	13
7.2.2 Capa de Acceso a Datos Relacional	13
7.2.3 Capa de Acceso a Datos Documental	13
7.2.4 Capa de Identidad	14
7.3 Flujo de Datos Detallado	14
7.4 APIs Internas	14
8. Arquitectura de Red	15
8.1 Topología General	15

8.2 Puertos y Servicios	15
8.3 NetworkPolicies Kubernetes (Zero-Trust)	16
NP-01: Default Deny All	16
NP-02: Allow External to Frontend	16
NP-03: Allow Frontend to SQL Server	16
NP-04: Allow Frontend to MongoDB	16
NP-05: Allow Frontend to LDAP	16
NP-06: Allow DNS Interno	16
8.4 Configuración de Firewall GFW	16
8.5 Red de Laboratorio de Sistemas Operativos (VirtualBox)	17
9. Diagramas UML	17
9.1 Diagrama de Casos de Uso (PlantUML)	17
9.2 Diagrama de Clases (PlantUML)	18
9.3 Diagrama de Secuencia — Login (PlantUML)	19
9.4 Diagrama de Componentes (PlantUML)	20
9.5 Diagrama de Despliegue (PlantUML)	20
10. Glosario Técnico	21
11. Manual de Despliegue	23
11.1 Requisitos Previos	23
11.2 Variables de Entorno y Secrets	23
11.3 Pasos de Instalación y Despliegue	23
11.4 Verificación del Despliegue	25
11.5 Acceso a MongoDB Shell (Queries Manuales)	25
12. Pruebas Documentadas	25
12.1 Estrategia de Testing	25
12.2 Casos de Prueba Funcionales	25
12.3 Pruebas de Seguridad de Red (NetworkPolicies)	26
12.4 Herramientas de Testing	27
13. Limitaciones Conocidas	27
14. Alcances y Límites del Sistema	28
14.1 Qué Cubre el Sistema	28
14.2 Qué NO Cubre el Sistema	28
14.3 Límites Operativos	29
15. Conclusión	29
15.1 Resultado Final del Proyecto	29
15.2 Beneficios Obtenidos	29
15.3 Posibles Mejoras Futuras	30

1. Introducción

1.1 Descripción General del Proyecto

El Ecosistema de Inventario Seguro (EIS) es una plataforma web de gestión de inventario tecnológico diseñada específicamente para los laboratorios de informática del Instituto Tecnológico Universitario (ITU). El sistema integra múltiples tecnologías de backend, dos motores de base de datos heterogéneos, autenticación federada, despliegue contenerizado y políticas de red de nivel empresarial, constituyendo así un ecosistema completo que cubre desde la capa de presentación hasta la infraestructura de seguridad perimetral.

El proyecto fue concebido como respuesta a la necesidad institucional de contar con una herramienta centralizada, segura y auditable para el seguimiento de activos tecnológicos. A diferencia de soluciones parciales o planillas de cálculo, el EIS provee una plataforma unificada que correlaciona datos administrativos (ubicación, responsable, mantenimiento) con datos técnicos (hardware, software, periféricos) mediante un identificador común de equipo.

1.2 Objetivo Principal

Desarrollar, contenerizar y desplegar en Minikube un ecosistema de microservicios que permita inventariar, consultar y gestionar el parque de computadoras de los laboratorios del ITU, garantizando autenticación centralizada mediante Active Directory/LDAP, persistencia dual en SQL Server y MongoDB, seguridad perimetral simulada con GFW/pfSense, y políticas de red Zero-Trust implementadas con NetworkPolicies de Kubernetes.

1.3 Problemática que Resuelve

El ITU carece de un sistema centralizado para el inventario de equipos de sus laboratorios. Los problemas concretos identificados son los siguientes:

- Dispersión de la información: los datos de ubicación, responsable y hardware de cada equipo se encuentran en registros independientes no correlacionados.
- Falta de trazabilidad: no existe historial de mantenimientos, asignaciones temporales o cambios de componentes.
- Ausencia de control de acceso granular: cualquier persona con acceso físico podría alterar registros.
- Vulnerabilidad de red: los sistemas actuales no contemplan políticas de segmentación ni principio de menor privilegio.
- Imposibilidad de auditoría: sin un sistema centralizado, no es posible determinar quién accedió a qué información ni cuándo.

1.4 Contexto del Sistema

El sistema opera en el entorno académico del ITU y se despliega sobre una infraestructura Minikube con CNF Calico. El contexto técnico abarca tres dominios diferenciados:

- Dominio de aplicación: aplicación web Flask/Python con interfaz Jinja2, accesible desde navegadores en la red del laboratorio.
- Dominio de datos: SQL Server (para datos relacionales de ubicación y asignación) y MongoDB (para documentos de hardware con estructura variable).
- Dominio de infraestructura: clúster Kubernetes en Minikube, firewall perimetral GFW, servidor de identidad OpenLDAP, repositorio Git versionado.

2. Decisiones de Diseño

2.1 Arquitectura Seleccionada: Microservicios Contenerizados

Se adopta una arquitectura de microservicios donde cada componente del sistema (frontend, base relacional, base documental y servicio de identidad) se ejecuta como un contenedor independiente dentro de un namespace de Kubernetes. Esta decisión se justifica en los siguientes puntos:

- Aislamiento de fallos: un error en la capa de base de datos no propaga su impacto al frontend.
- Escalabilidad horizontal: cada microservicio puede escalarse de forma independiente sin afectar a los demás.
- Trazabilidad y auditabilidad: los logs de cada contenedor son independientes y consultables por separado.
- Facilidad de despliegue reproducible: los manifiestos de Kubernetes garantizan que el entorno de desarrollo, prueba y producción son idénticos.

2.2 Tecnologías Elegidas y Justificación

Componente	Tecnología Elegida	Alternativas Consideradas	Justificación
Backend/Frontend	Python 3 + Flask + Jinja2	Node.js + Express, Spring Boot	Menor curva de aprendizaje, conectividad nativa a pymysql, pymongo y python-ldap, contenedor más liviano que JVM
Base Relacional	SQL Server Express 2022	MySQL 8, PostgreSQL 15	Compatibilidad con entorno Windows Server institucional, integración nativa con AD, familiaridad del equipo
Base Documental	MongoDB 7	CouchDB, Elasticsearch	Esquema flexible para hardware variado, mongosh

			accesible para queries manuales, imagen Docker oficial
Autenticación	OpenLDAP	Active Directory real, Keycloak	Liviano, sin licencia, contenedorizable, protocolo LDAP compatible con python-ldap
Orquestación	Kubernetes/Minikube	Docker Compose, k3s	Requerido por la consigna, permite NetworkPolicies reales
CNI de Red	Calico	Flannel, Weave	Único CNI que aplica NetworkPolicies correctamente en Minikube
Firewall	GFW (UFW)	pfSense, iptables directo	Interfaz gráfica disponible, menor complejidad de configuración en lab
Versionado	Git + repositorio unificado	SVN, carpetas compartidas	Estándar de la industria, permite historial de commits por integrante

2.3 Justificación de la Base de Datos Dual

La decisión de utilizar dos motores de base de datos heterogéneos responde a la naturaleza diferenciada de los datos manejados por el sistema:

2.3.1 SQL Server — Datos Estructurados (ubicacion-db)

Los datos de ubicación, asignación y mantenimiento tienen una estructura fija y relaciones claras entre entidades (aulas, mesas, equipos, personas, fechas). SQL Server es el motor adecuado para este tipo de datos porque:

- Garantiza integridad referencial mediante claves foráneas.
- Permite consultas JOIN eficientes para correlacionar equipos con responsables y aulas.
- Ofrece soporte nativo para transacciones ACID.
- Se integra con autenticación de Windows/AD para usuarios de base de datos.

2.3.2 MongoDB — Datos Documentales (inventario-db)

Los datos de hardware de cada computadora presentan variabilidad estructural: una laptop tiene batería, un servidor puede tener múltiples discos RAID, una workstation puede tener GPU dedicada. MongoDB es el motor adecuado porque:

- Almacena documentos JSON con estructura variable sin necesidad de migraciones de esquema.
- Permite campos opcionales nativamente (por ejemplo, el campo 'bateria' existe solo en laptops).

- Facilita el acceso desde shell con mongosh para queries manuales requeridas por la consigna.
- Escala horizontalmente con sharding si el parque de equipos crece significativamente.

2.4 Patrones de Diseño Aplicados

Patrón	Aplicación en el Proyecto
Repository Pattern	La capa de acceso a datos separa la lógica de negocio de las consultas SQL y MongoDB. Módulos db_sql.py y db_mongo.py independientes.
Service Layer	La lógica de negocio (correlación SQL+MongoDB por id_equipo) reside en una capa de servicios, no en las rutas Flask.
Factory Method	Los objetos de conexión a base de datos se instancian mediante funciones factory que leen variables de entorno.
Decorator Pattern	@login_required aplicado en Flask para proteger todas las rutas que requieren autenticación.
MVC Adaptado	Flask routes = Controllers, Jinja2 templates = Views, modelos Python = Models (sin ORM completo por decisión de simplicidad).

2.5 Decisiones de Seguridad

- Zero-Trust Network: se aplica deny-all por defecto en el namespace y se permiten solo las rutas de comunicación estrictamente necesarias.
- Secrets de Kubernetes: contraseñas, cadenas de conexión y credenciales LDAP se almacenan como objetos Secret, nunca en variables de entorno planas ni en código fuente.
- Principio de Menor Privilegio: cada pod recibe solo los permisos de red necesarios para cumplir su función.
- Autenticación Centralizada: ningún servicio mantiene base de usuarios propia; toda autenticación se delega a OpenLDAP.
- Sin exposición de bases de datos: SQL Server y MongoDB tienen Services de tipo ClusterIP, inaccesibles desde fuera del cluster.

2.6 Ventajas y Desventajas del Diseño

Aspecto	Ventajas	Desventajas
Microservicios	Aislamiento, escalabilidad independiente, despliegue parcial	Mayor complejidad operativa, overhead de red entre pods
Base Dual	Modelo de datos óptimo para cada tipo de información	Mayor superficie de mantenimiento, consistencia eventual entre bases

Flask	Rápido desarrollo, liviano, biblioteca Python amplia	No tan adecuado para alta concurrencia sin ASGI (Gunicorn recomendado)
Minikube	Entorno local reproducible, gratuito	No recomendado para producción real, recursos limitados del host
OpenLDAP vs AD real	Sin licencia, contenedorizable, simple	No simula la complejidad real de Active Directory institucional

3. Roles del Proyecto

3.1 Estructura del Equipo

El proyecto es desarrollado por un equipo de 5 integrantes, cada uno con un rol técnico definido y una responsabilidad de defensa específica. La asignación de roles sigue una distribución que cubre la totalidad de las capas del ecosistema.

Rol	Responsable	Área de Dominio	Entregable Principal
Rol 1 — Arquitecto / Líder Técnico	Integrante 1	Arquitectura, Integración, Documentación	Diagrama de arquitectura, repositorio Git unificado, documentación técnica, presentación PPT
Rol 2 — Ingeniero de Base de Datos Relacional	Integrante 2	SQL Server, Modelo Relacional	Script DDL completo, datos de prueba, manifiesto ubicacion-db-deployment.yaml
Rol 3 — Ingeniero de Base de Datos Documental	Integrante 3	MongoDB, NoSQL	Documentos JSON, queries mongosh, manifiesto inventario-db-deployment.yaml
Rol 4 — SysAdmin de Identidad	Integrante 4	OpenLDAP, Estructura de Directorios, Roles RBAC.	Estructura jerárquica de usuarios y grupos organizacionales.
Rol 5 — Ing. Cloud y Perímetro	Integrante 5	Kubernetes, CNI Calico, NetworkPolicies, GFW.	Manifiestos YAML de aislamiento de red y reglas de firewall del host.
Rol 6 — Desarrollador de Aplicación Web	Integrante 6	Frontend Visual (Tailwind), Jinja2, Matriz de Pruebas de Red.	Vistas HTML dinámicas securizadas desde servidor y ejecución de pruebas en vivo.

3.2 Metodología de Trabajo

El equipo adopta una metodología ágil simplificada basada en Scrum, adaptada a la duración y naturaleza académica del proyecto. Las ceremonias principales son:

- Sprint Planning semanal: distribución de tareas por rol, identificación de dependencias y definición de criterios de aceptación.
- Daily Standup (15 min): sincronización diaria del estado de cada módulo y bloqueos detectados.
- Sprint Review: demostración del ecosistema funcional al finalizar cada iteración de dos semanas.
- Retrospectiva: identificación de mejoras en el proceso de integración.

3.3 Flujo de Trabajo Git

El repositorio Git unificado sigue un flujo de trabajo estructurado basado en ramas de características (*feature branches*) asignadas por rol técnico. Esta segmentación de ramas mitiga el riesgo de conflictos masivos (*merge conflicts*) en el código y asegura el aislamiento de responsabilidades durante la etapa de desarrollo paralelo:

- **main**: Rama principal y estable de producción. Está protegida contra escrituras directas y solo recibe integraciones (*merges*) consolidadas y revisadas de forma estricta por el Líder Técnico (Rol 1).
- **feature/flask-backend**: Rama asignada al Líder Técnico (Rol 1). Contiene el núcleo de la lógica de negocio, la configuración de los controladores de persistencia cruzada y las rutas de API del servidor Flask.
- **feature/sql-db**: Rama asignada al DBA Relacional (Rol 2). Destinada exclusivamente al desarrollo de los scripts DDL de creación de estructuras, inserción de datos administrativos estáticos y al armado del manifiesto Kubernetes correspondiente a [ubicacion-db](#).
- **feature/mongo-db**: Rama asignada al DBA Documental (Rol 3). Contiene los esquemas de colecciones dinámicas BSON, los scripts de inicialización de hardware ([seed.js](#)) y el manifiesto Kubernetes para el despliegue de [inventario-db](#).
- **feature/ldap-identity**: Rama asignada al SysAdmin de Identidad (Rol 4). Dedicada a la configuración del árbol jerárquico de directorios, usuarios institucionales y asignación de grupos de OpenLDAP en el microservicio [ldap-service](#).
- **feature/cloud-infra**: Rama asignada al Ingeniero Cloud (Rol 5). Concentra la orquestación global en Kubernetes, la implementación estricta de las seis (6) [NetworkPolicies](#) bajo Calico y el aprovisionamiento de las políticas perimetrales del host (GUPFW).
- **feature/ui-tailwind**: Rama asignada al Desarrollador UI y QA Tester (Rol 6). Diseñada para la maquetación visual del entorno oscuro (*dark mode*), la configuración de estilos mediante Tailwind CSS y la inyección de datos dinámicos en las plantillas del motor Jinja2.

- **feature/qa-testing**: Rama compartida de control de calidad (Rol 6 y Rol 5). Contiene los scripts de automatización de pruebas de red internas (kubect`l` `exec` cruzados) y la documentación de la matriz de verificación de vulnerabilidades exigida para la demo en vivo.

4. Requisitos Funcionales

Los siguientes requisitos funcionales definen las capacidades que el sistema debe proveer. Cada requisito es trazable a un caso de uso y a uno o más componentes de la arquitectura.

ID	Requisito Funcional	Prioridad	Componente
RF-01	El sistema deberá permitir al usuario autenticarse mediante credenciales validadas contra el servidor OpenLDAP institucional. Sin autenticación exitosa no se podrá acceder a ninguna funcionalidad.	Alta	inventario-web / ldap-service
RF-02	El sistema deberá mostrar un listado completo de todos los equipos del inventario, incluyendo su ubicación (aula, laboratorio, número de banco/mesa), responsable asignado y fecha de último mantenimiento, con datos provenientes de SQL Server.	Alta	inventario-web / ubicacion-db
RF-03	El sistema deberá presentar una vista de detalle unificada por equipo, combinando en una sola pantalla los datos de ubicación de SQL Server y los componentes de hardware de MongoDB, correlacionados por el campo <code>id_equipo</code> .	Alta	inventario-web / ubicacion-db / inventario-db
RF-04	El sistema deberá permitir crear nuevos registros de hardware en MongoDB con los campos: <code>id_equipo</code> , fabricante, modelo, tipo (desktop/laptop), CPU, RAM, disco(s), sistema operativo, monitor, mouse y teclado.	Alta	inventario-web / inventario-db
RF-05	El sistema deberá permitir actualizar los datos de hardware de un equipo existente en MongoDB, modificando uno o más campos del documento sin necesidad de reemplazar el documento completo.	Alta	inventario-web / inventario-db
RF-06	El sistema deberá permitir eliminar el registro de hardware de un equipo específico de MongoDB, requiriendo confirmación explícita del usuario antes de ejecutar la operación.	Media	inventario-web / inventario-db
RF-07	El sistema deberá permitir crear nuevos registros de ubicación/asignación en SQL Server, incluyendo la asociación entre un equipo, un aula/laboratorio y un responsable técnico.	Alta	inventario-web / ubicacion-db

RF-08	El sistema deberá permitir actualizar la información de ubicación y asignación de un equipo en SQL Server, registrando el cambio con timestamp.	Alta	inventario-web / ubicacion-db
RF-09	La base de datos MongoDB deberá ser accesible desde la línea de comandos del contenedor (mongosh) para permitir la ejecución manual de queries de búsqueda filtrada, actualización y eliminación.	Alta	inventario-db
RF-10	El sistema deberá permitir cerrar la sesión del usuario, invalidando la sesión activa y redirigiendo a la pantalla de login.	Media	inventario-web
RF-11	El sistema deberá diferenciar las vistas y operaciones disponibles según el rol del usuario autenticado (Administrador, Técnico o Docente/Alumno), obteniendo el rol desde los atributos del usuario en OpenLDAP.	Alta	inventario-web / ldap-service
RF-12	El sistema deberá registrar en log las operaciones de autenticación (login exitoso, login fallido) y las operaciones de escritura sobre el inventario.	Media	inventario-web

5. Requisitos No Funcionales

ID	Categoría	Requisito No Funcional
RNF-01	Seguridad Perimetral	Todo el tráfico entrante al ecosistema deberá pasar obligatoriamente por la IP autorizada definida en las reglas de GFW. Se bloqueará cualquier acceso directo a bases de datos o servicios internos desde la red pública.
RNF-02	Seguridad Interna (Zero-Trust)	Las NetworkPolicies de Kubernetes deberán garantizar que cada servicio solo acepta conexiones de los pods explícitamente autorizados. El tráfico no autorizado dentro del namespace debe ser denegado por defecto.
RNF-03	Contenerización Total	Todos los servicios del ecosistema (inventario-web, ubicacion-db, inventario-db, ldap-service) deberán ejecutarse como contenedores Docker dentro del clúster Minikube.
RNF-04	Principio de Menor Privilegio	Cada servicio deberá tener exclusivamente los permisos de red, sistema de archivos y base de datos necesarios para cumplir su función específica. Los contenedores no deberán ejecutarse como root.
RNF-05	Versionado de Código	Todo el código fuente, manifiestos de Kubernetes, scripts de base de datos y documentación deberán estar versionados en un repositorio Git con historial de commits que evidencie la contribución de cada integrante.

RNF-06	Disponibilidad en Defensa	El ecosistema completo deberá poder levantarse desde cero en la PC del laboratorio mediante un script de arranque de un solo comando en menos de 5 minutos.
RNF-07	Rendimiento	La respuesta de la vista de detalle de equipo (que combina una consulta SQL y una consulta MongoDB) no deberá superar los 2 segundos en condiciones de laboratorio normal.
RNF-08	Escalabilidad	La arquitectura deberá permitir aumentar el número de réplicas del servicio inventario-web mediante kubectl scale sin modificar el código de la aplicación.
RNF-09	Compatibilidad	La aplicación web deberá ser accesible desde los navegadores Google Chrome, Mozilla Firefox y Microsoft Edge en sus versiones actuales, sin requerir plugins ni extensiones adicionales.
RNF-10	Usabilidad	La interfaz de usuario deberá permitir a un técnico sin formación previa en el sistema localizar y visualizar el detalle de un equipo en menos de 3 clicks desde el login.
RNF-11	Mantenibilidad	El código de la aplicación deberá seguir las convenciones PEP 8 para Python. Los manifiestos de Kubernetes deberán incluir comentarios explicativos en sus secciones principales.
RNF-12	Confidencialidad de Credenciales	Ninguna contraseña, token o cadena de conexión deberá estar hardcodeda en el código fuente ni en variables de entorno en texto plano. Se deberán utilizar Kubernetes Secrets para toda información sensible.

6. Casos de Uso

6.1 Actores del Sistema

Actor	Tipo	Descripción
Administrador	Principal (humano)	Usuario con acceso total al sistema. Puede ver, crear, editar y eliminar cualquier registro de inventario. Gestiona usuarios y configuraciones. Sus credenciales tienen atributo memberOf=Administradores en OpenLDAP.
Técnico	Principal (humano)	Usuario con acceso parcial. Puede consultar equipos y actualizar datos de hardware en MongoDB, pero no puede eliminar registros ni modificar ubicaciones. Atributo LDAP: memberOf=Técnicos.
Docente / Alumno	Principal (humano)	Usuario de solo lectura. Puede consultar el equipo que tiene asignado temporalmente y ver sus especificaciones técnicas. No puede modificar ningún dato.
OpenLDAP	Sistema externo	Servidor de autenticación que valida credenciales y provee atributos de usuario (grupos, roles) al sistema.
SQL Server	Sistema externo	Motor de base de datos relacional que persiste datos de ubicación, asignación y mantenimiento.

MongoDB	Sistema externo	Motor de base de datos documental que persiste datos técnicos de hardware de los equipos.
---------	-----------------	---

6.2 Casos de Uso Detallados

CU-01: Autenticación de Usuario

Campo	Detalle
Caso de Uso	CU-01 — Autenticación de Usuario
Actores	Administrador, Técnico, Docente/Alumno
Precondiciones	El servicio ldap-service está disponible. El usuario posee credenciales válidas en OpenLDAP.
Postcondiciones	El usuario tiene una sesión activa con rol asignado. Se registra el evento de login en el log del sistema.
Flujo Principal	1. El usuario accede a la URL de la aplicación. 2. El sistema presenta el formulario de login. 3. El usuario ingresa usuario y contraseña. 4. La aplicación realiza bind LDAP con las credenciales. 5. OpenLDAP valida y devuelve los atributos del usuario. 6. La aplicación crea la sesión con el rol correspondiente. 7. El sistema redirige al usuario al listado de equipos.
Flujo Alternativo FA-01	Si las credenciales son incorrectas: OpenLDAP rechaza el bind. La aplicación muestra el mensaje 'Credenciales inválidas'. Se registra el intento fallido en el log. El sistema vuelve al formulario de login.
Flujo Alternativo FA-02	Si ldap-service no está disponible: La aplicación captura la excepción de conexión. Muestra el mensaje 'Servicio de autenticación no disponible'. No permite el acceso.

CU-02: Consulta de Inventario (Listado)

Campo	Detalle
Caso de Uso	CU-02 — Consulta de Inventario
Actores	Administrador, Técnico, Docente/Alumno (vista restringida)
Precondiciones	El usuario tiene sesión activa. Los servicios ubicacion-db e inventario-web están disponibles.
Postcondiciones	El usuario visualiza el listado de equipos filtrado según su rol.
Flujo Principal	1. El usuario autenticado accede a /equipos. 2. La aplicación consulta SELECT * FROM equipos JOIN aulas JOIN responsables en SQL Server. 3. El sistema renderiza la tabla de equipos con columnas: Código, Aula, Laboratorio, Mesa, Responsable, Estado, Último Mantenimiento. 4. El usuario puede filtrar por aula, laboratorio o estado.
Flujo Alternativo	Si el rol es Docente/Alumno: el sistema filtra automáticamente por usuario_asignado = usuario_logueado. Solo ve el equipo que tiene asignado.

CU-03: Consulta de Detalle de Equipo

Campo	Detalle
Caso de Uso	CU-03 — Consulta de Detalle de Equipo
Actores	Administrador, Técnico, Docente/Alumno
Precondiciones	El usuario tiene sesión activa. Los servicios ubicacion-db, inventario-db e inventario-web están disponibles.
Postcondiciones	El usuario visualiza la ficha completa del equipo combinando datos de ambas bases.
Flujo Principal	1. El usuario selecciona un equipo del listado. 2. La aplicación extrae el id_equipo del registro SQL. 3. Simultáneamente consulta SQL Server (datos administrativos) y MongoDB (datos técnicos) usando ese id_equipo. 4. La aplicación combina los resultados en un objeto unificado. 5. El sistema renderiza la vista de detalle con secciones: Ubicación, Asignación, Mantenimiento y Hardware.
Flujo Alternativo	Si no existe documento en MongoDB para ese id_equipo: La sección de hardware muestra 'Sin datos de hardware registrados'. El sistema no genera error.

CU-04: Gestión CRUD de Hardware (MongoDB)

Campo	Detalle
Caso de Uso	CU-04 — Gestión de Hardware
Actores	Administrador (C/R/U/D), Técnico (R/U)
Precondiciones	El usuario tiene sesión activa con rol Administrador o Técnico. El servicio inventario-db está disponible.
Postcondiciones	El documento de hardware en MongoDB refleja los cambios. Se registra la operación en el log.
Flujo — Crear	Administrador accede a /hardware/nuevo. Completa el formulario con los campos de hardware. La aplicación ejecuta db.hardware.insertOne({...}) en MongoDB. El sistema confirma la inserción y redirige al detalle.
Flujo — Actualizar	Técnico o Admin accede a /hardware/{id}/editar. Modifica los campos necesarios. La aplicación ejecuta db.hardware.updateOne({id_equipo: id}, {\$set: {...}}). El sistema confirma la actualización.
Flujo — Eliminar (solo Admin)	Admin accede al detalle del equipo. Selecciona 'Eliminar hardware'. El sistema solicita confirmación. La aplicación ejecuta db.hardware.deleteOne({id_equipo: id}). El sistema redirige al listado.

7. Arquitectura del Proyecto

7.1 Visión General

El EIS implementa una arquitectura de microservicios de cuatro capas, cada una correspondiente a un servicio contenerizado dentro del namespace inventario-seguro de Kubernetes. La separación de responsabilidades es estricta: el único componente que puede comunicarse con las bases de datos y el servidor de identidad es el servicio de aplicación (inventario-web).

7.2 Capas de la Arquitectura

7.2.1 Capa de Presentación (Frontend)

Tecnología: Flask + Jinja2 (Python 3.11). El servicio inventario-web es el único componente expuesto al exterior. Maneja sesiones de usuario, renderizado de templates HTML y coordinación de consultas a las capas de datos. Expone el puerto 5000/TCP internamente y es accesible desde el exterior mediante un Service NodePort o LoadBalancer (con minikube tunnel).

7.2.2 Capa de Acceso a Datos Relacional

Tecnología: SQL Server Express 2022 (imagen mcr.microsoft.com/mssql/server:2022-latest). El servicio ubicacion-db persiste todas las entidades relacionales: aulas, laboratorios, equipos, responsables, asignaciones y mantenimientos. Expone el puerto 1433/TCP solo dentro del namespace, accesible únicamente desde el pod del frontend.

7.2.3 Capa de Acceso a Datos Documental

Tecnología: MongoDB 7 (imagen mongo:7). El servicio inventario-db almacena documentos JSON de hardware con estructura variable. Expone el puerto 27017/TCP únicamente dentro del namespace. Es accesible desde el shell del contenedor mediante mongosh para ejecución manual de queries.

7.2.4 Capa de Identidad

Tecnología: OpenLDAP (imagen osixia/openldap). El servicio ldap-service centraliza la autenticación de todos los usuarios del sistema. Expone el puerto 389/TCP (LDAP) o 636/TCP (LDAPS) solo dentro del namespace, accesible únicamente desde el frontend.

7.3 Flujo de Datos Detallado

El siguiente diagrama de texto representa el flujo completo de datos para una consulta de detalle de equipo:

FLUJO DE CONSULTA DE DETALLE DE EQUIPO

```

[Navegador]
|
| HTTP GET /equipo/{id_equipo}
v
[GUPFW Firewall] --> valida IP origen autorizada
|
v
[inventario-web:5000]
|
|--[1]--> LDAP bind check: sesion valida?
|         [ldap-service:389]
|
|--[2]--> SELECT equipo, aula, responsable WHERE id={id}
|         [ubicacion-db:1433] <-- resultado: datos_sql
|
|--[3]--> db.hardware.findOne({id_equipo: id})
|         [inventario-db:27017] <-- resultado: datos_mongo
|
|--[4]--> combina datos_sql + datos_mongo
|
v
Renderiza template detalle.html con datos unificados
|
v
[Navegador] <-- respuesta HTTP 200 con vista completa

```

7.4 APIs Internas

La aplicación Flask expone las siguientes rutas HTTP principales:

Método	Ruta	Descripción	Autenticación
GET/ POST	/login	Formulario de login y procesamiento de credenciales LDAP	No requerida
GET	/logout	Cierre de sesión y limpieza de cookie	Requerida
GET	/equipos	Listado paginado de equipos con filtros	Requerida
GET	/equipo/<id>	Vista de detalle unificado SQL+MongoDB	Requerida
GET/ POST	/equipo/nuevo	Formulario para crear equipo (SQL+Mongo)	Admin
GET/ POST	/equipo/<id>/editar	Edición de datos de ubicación y hardware	Admin/Técnico
POST	/equipo/<id>/eliminar	Eliminación de registros (ambas bases)	Admin
GET	/health	Endpoint de healthcheck para Kubernetes readinessProbe	No requerida

8. Arquitectura de Red

8.1 Topología General

El ecosistema opera en tres zonas de red diferenciadas, que simulan la arquitectura DMZ típica de un entorno empresarial:

- Zona Pública (Red del Laboratorio): desde donde acceden los usuarios mediante navegador. La IP del host Minikube actúa como punto de entrada.
- DMZ Simulada: representada por las reglas GFW en el host, que permiten solo el tráfico hacia el puerto expuesto por Minikube (NodePort o LoadBalancer).
- Red Privada (Namespace Kubernetes): zona interna donde operan todos los microservicios. El acceso entre pods está controlado por NetworkPolicies de Calico.

8.2 Puertos y Servicios

Servicio	Puerto Interno	Puerto Externo	Protocolo	Tipo Service K8s	Acceso
inventario-web	5000	30080 (NodePort)	TCP/HTTP	NodePort	Público (filtrado por GFW)
ubicacion-db (SQL Server)	1433	—	TCP	ClusterIP	Solo desde inventario-web
inventario-db (MongoDB)	27017	—	TCP	ClusterIP	Solo desde inventario-web
ldap-service (OpenLDAP)	389 / 636	—	TCP	ClusterIP	Solo desde inventario-web

8.3 NetworkPolicies Kubernetes (Zero-Trust)

Se implementa el modelo Zero-Trust mediante las siguientes NetworkPolicies aplicadas en el namespace inventario-seguro:

NP-01: Default Deny All

Política base que deniega todo el tráfico de entrada (ingress) y salida (egress) para todos los pods del namespace. Todas las demás políticas son excepciones explícitas a esta regla.

NP-02: Allow External to Frontend

Permite el tráfico entrante hacia el pod inventario-web desde cualquier origen en el puerto 5000/TCP. Esta es la única regla que permite tráfico desde fuera del namespace.

NP-03: Allow Frontend to SQL Server

Permite que los pods con label app=inventario-web envíen tráfico al puerto 1433/TCP de los pods con label app=ubicacion-db. Bloquea cualquier otro origen.

NP-04: Allow Frontend to MongoDB

Permite que los pods con label app=inventario-web envíen tráfico al puerto 27017/TCP de los pods con label app=inventario-db. Bloquea cualquier otro origen.

NP-05: Allow Frontend to LDAP

Permite que los pods con label app=inventario-web envíen tráfico al puerto 389/TCP de los pods con label app=ldap-service. Bloquea cualquier otro origen.

NP-06: Allow DNS Interno

Permite a todos los pods del namespace resolver nombres mediante DNS interno de Kubernetes (puerto 53/UDP y 53/TCP) hacia el namespace kube-system.

8.4 Configuración de Firewall GUFW

En el host que ejecuta Minikube, GUFW (interfaz gráfica de UFW) se configura con las siguientes reglas:

Regla	Dirección	Protocolo	Puerto	Origen	Acción
FW-01	ENTRADA	TCP	30080	192.168.100.0/24 (red laboratorio)	ALLOW
FW-02	ENTRADA	TCP	22	192.168.100.0/24	ALLOW (admin SSH)
FW-03	ENTRADA	TCP/UDP	CUALQUIER	CUALQUIER	DENY (default)
FW-04	SALIDA	TCP/UDP	CUALQUIER	CUALQUIER	ALLOW

8.5 Red de Laboratorio de Sistemas Operativos (VirtualBox)

Adicionalmente, en el contexto de la materia Sistemas Operativos, se trabaja con una red virtual en VirtualBox compuesta por tres máquinas virtuales:

VM	SO	Interfaz	Red	IP Configurada	Servicios
----	----	----------	-----	----------------	-----------

Router	Linux Alpine	eth0	NAT/Internet	10.0.2.15/24 (DHCP)	Routing, DHCP Relay
Router	Linux Alpine	eth1	Server_Network	192.168.100.254/24	Gateway servidores
Router	Linux Alpine	eth2	Client_Network	192.168.200.254/24	Gateway clientes
Server (DC01)	Windows Server 2022	NIC1	Server_Network	192.168.100.10/24	AD DS, DNS, DHCP, GPO, IIS, Server
Client	Windows 11 Pro	NIC1	Client_Network	192.168.200.10/24	Unido al dominio ITU.local

9. Diagramas UML

Los siguientes diagramas se representan en notación PlantUML/texto estructurado. Para renderizarlos visualmente, se recomienda usar <https://plantuml.com> o la extensión PlantUML para VS Code.

9.1 Diagrama de Casos de Uso (PlantUML)

```
@startuml
CasosDeUso
left to right direction
actor Administrador as Admin
actor Tecnico as Tec
actor "Docente/Alumno" as Doc
actor OpenLDAP

rectangle "EIS - Ecosistema de Inventario Seguro" {
    usecase "Autenticarse" as UC1
    usecase "Ver listado de equipos" as UC2
    usecase "Ver detalle de equipo" as UC3
    usecase "Crear hardware (MongoDB)" as UC4
    usecase "Actualizar hardware" as UC5
    usecase "Eliminar hardware" as UC6
    usecase "Gestionar ubicacion (SQL)" as UC7
    usecase "Cerrar sesion" as UC8
}

Admin --> UC1 ; Admin --> UC2 ; Admin --> UC3
Admin --> UC4 ; Admin --> UC5 ; Admin --> UC6
Admin --> UC7 ; Admin --> UC8
Tec --> UC1 ; Tec --> UC2 ; Tec --> UC3
Tec --> UC5 ; Tec --> UC8
Doc --> UC1 ; Doc --> UC3 ; Doc --> UC8
UC1 --> OpenLDAP : <<include>>
@enduml
```

9.2 Diagrama de Clases (PlantUML)

```
@startuml Clases
class Equipo {
    +id_equipo: String
    +codigo_inventario: String
    +estado: String
    +getUbicacion(): Ubicacion
    +getHardware(): Hardware
}

class Ubicacion {
    +id_ubicacion: Int
    +id_equipo: String
    +aula: String
    +laboratorio: String
    +numero_banco: String
    +fecha_ultimo_mant: Date
    +fecha_proximo_mant: Date
}

class Asignacion {
    +id_asignacion: Int
    +id_equipo: String
    +responsable_tecnico: String
    +usuario_asignado: String
    +rol_asignado: String
    +fecha_desde: Date
    +fecha_hasta: Date
}

class Hardware {
    +id_equipo: String
    +fabricante: String
    +modelo: String
    +tipo: String
    +cpu: CPU
    +ram: RAM
    +discos: List<Disco>
    +sistema_operativo: String
    +perifericos: Perifericos
}

class CPU { +marca +modelo +generacion }
class RAM { +cantidad_gb +tipo }
class Disco { +tipo +capacidad_gb }
class Perifericos { +monitor +teclado +mouse }
class Usuario { +uid +nombre +email +rol +grupo_ldap }
class Aula { +id_aula +nombre +laboratorio }
class Responsable { +id_responsable +nombre +email }

Equipo "1" -- "1" Ubicacion
Equipo "1" -- "*" Asignacion
Equipo "1" -- "1" Hardware
Hardware "1" *-- "1" CPU
Hardware "1" *-- "1" RAM
Hardware "1" *-- "*" Disco
Hardware "1" *-- "1" Perifericos
Ubicacion "*" --> "1" Aula
```

```
Asignacion "*" --> "1" Responsable
@enduml
```

9.3 Diagrama de Secuencia — Login (PlantUML)

```
@startuml SecuenciaLogin
actor Usuario
participant "inventario-web\n(Flask)" as WEB
participant "ldap-service\n(OpenLDAP)" as LDAP
database "Sesion\n(Flask Session)" as SES

Usuario -> WEB: GET /login
WEB -> Usuario: Muestra formulario login
Usuario -> WEB: POST /login {usuario, password}
WEB -> LDAP: ldap.bind(uid=usuario, password=password)
alt Credenciales validas
    LDAP -> WEB: Bind exitoso + atributos (grupos, rol)
    WEB -> SES: session['user'] = {uid, rol, nombre}
    WEB -> Usuario: Redirige a /equipos (302)
else Credenciales invalidas
    LDAP -> WEB: LDAPBindError
    WEB -> WEB: Registra intento fallido en log
    WEB -> Usuario: Muestra error 'Credenciales invalidas'
else Servicio no disponible
    LDAP -> WEB: ConnectionError
    WEB -> Usuario: Muestra error 'Servicio no disponible'
end
@enduml
```

9.4 Diagrama de Componentes (PlantUML)

```
@startuml Componentes
package "Namespace: inventario-seguro" {

    component [inventario-web\nFlask:5000] as WEB
    component [ubicacion-db\nSQL Server:1433] as SQL
    component [inventario-db\nMongoDB:27017] as MONGO
    component [ldap-service\nOpenLDAP:389] as LDAP

    WEB --> SQL : pymssql / port 1433
    WEB --> MONGO : pymongo / port 27017
    WEB --> LDAP : python-ldap / port 389
}

cloud "Red Laboratorio" {
    [Navegador] as NAV
    [GFW Firewall] as FW
}

NAV --> FW : HTTP
FW --> WEB : HTTP:30080 -> 5000 (NodePort)
@enduml
```

9.5 Diagrama de Despliegue (PlantUML)

```
@startuml Despliegue
node "Host PC Laboratorio" {
    node "Minikube (CNI: Calico)" {
        node "Namespace: inventario-seguro" {
            artifact "Pod: inventario-web" as WPOD
            artifact "Pod: ubicacion-db" as SQLPOD
            artifact "Pod: inventario-db" as MPOD
            artifact "Pod: ldap-service" as LPOD
            artifact "NetworkPolicy: deny-all" as NP1
            artifact "NetworkPolicy: allow-web" as NP2
        }
        artifact "Service: web-svc (NodePort)" as NSVC
        artifact "Service: sql-svc (ClusterIP)" as Ssvc
        artifact "Service: mongo-svc (ClusterIP)" as MSVC
        artifact "Service: ldap-svc (ClusterIP)" as LSVC
    }
    artifact "GFW Firewall"
}

node "Red Laboratorio" {
    node "PC Cliente"
}

"PC Cliente" --> "GFW Firewall" : HTTP
"GFW Firewall" --> NSVC : TCP:30080
NSVC --> WPOD
WPOD --> Ssvc --> SQLPOD
WPOD --> MSVC --> MPOD
WPOD --> LSVC --> LPOD
@enduml
```

10. Glosario Técnico

Término	Definición
Active Directory (AD)	Servicio de directorio de Microsoft que centraliza la autenticación, autorización y administración de usuarios, equipos y recursos en una red Windows mediante el protocolo LDAP y Kerberos.
Calico	Plugin CNI (Container Network Interface) para Kubernetes que implementa la aplicación efectiva de NetworkPolicies mediante eBPF o iptables. Requerido en Minikube para que las políticas de red tengan efecto real.
ClusterIP	Tipo de Service de Kubernetes que expone un pod únicamente dentro del cluster, sin acceso externo. Utilizado para bases de datos y LDAP en este proyecto.
CNI (Container Network Interface)	Especificación y conjunto de plugins que configuran la red de los contenedores en Kubernetes. Determina cómo se asignan IPs y cómo se aplican las NetworkPolicies.

ConfigMap	Objeto de Kubernetes que almacena datos de configuración no sensibles en formato clave-valor. Utilizado para variables de entorno que no requieren confidencialidad.
Deployment	Objeto de Kubernetes que define y gestiona el ciclo de vida de un conjunto de Pods idénticos, incluyendo rollouts, rollbacks y escalabilidad.
DMZ (Demilitarized Zone)	Zona de red intermedia entre la red pública e internet y la red interna privada, donde se ubican los servicios accesibles externamente (como el frontend), sin exponer directamente los sistemas internos.
Flask	Microframework web para Python, ligero y extensible, que permite crear aplicaciones web y APIs REST con mínima configuración inicial.
GUFW	Graphical Uncomplicated Firewall. Interfaz gráfica para UFW (Uncomplicated Firewall), que a su vez es una interfaz simplificada para iptables en Linux.
Jinja2	Motor de templates para Python, utilizado por Flask para renderizar páginas HTML dinámicas con variables, bucles y estructuras de control.
Kerberos	Protocolo de autenticación de red que utiliza tickets cifrados para verificar la identidad de usuarios y servicios, sin transmitir contraseñas en texto plano. Protocolo base de Active Directory.
LDAP (Lightweight Directory Access Protocol)	Protocolo estándar de la industria para acceder y mantener servicios de directorio distribuidos. Utilizado para autenticación centralizada de usuarios.
Minikube	Herramienta que permite ejecutar un clúster de Kubernetes de un solo nodo en una máquina local o de laboratorio. Utilizada en este proyecto como entorno de despliegue.
mongosh	Shell interactiva oficial de MongoDB que permite ejecutar queries JavaScript directamente contra el motor de base de datos. Requerida por la consigna para acceso manual.
Namespace (Kubernetes)	Mecanismo de aislamiento lógico dentro de un clúster Kubernetes que permite separar recursos, aplicar límites y políticas a un conjunto específico de pods y servicios.
NetworkPolicy	Objeto de Kubernetes que define reglas de firewall a nivel de pod, especificando qué tráfico de red está permitido entre pods y hacia/desde el exterior del namespace.
NodePort	Tipo de Service de Kubernetes que expone un pod en un puerto fijo del nodo (entre 30000-32767), permitiendo acceso externo al servicio.
OpenLDAP	Implementación de código abierto del protocolo LDAP, utilizada en este proyecto como alternativa a Active Directory real para autenticación centralizada en entorno de laboratorio.
PersistentVolume (PV)	Objeto de Kubernetes que representa almacenamiento persistente del cluster, independiente del ciclo de vida del Pod.

PersistentVolumeClaim (PVC)	Solicitud de almacenamiento persistente realizada por un Pod. Permite que los datos de las bases de datos sobrevivan reinicios del contenedor.
pfSense	Distribución de firewall/router basada en FreeBSD, de código abierto, con interfaz web de administración. Mencionada en la consigna como alternativa a GFW.
Pod	Unidad mínima de despliegue en Kubernetes. Contiene uno o más contenedores que comparten red y almacenamiento.
pymongo	Driver oficial de Python para conectarse y operar con bases de datos MongoDB.
pymssql	Driver de Python para conectarse a SQL Server mediante el protocolo TDS.
python-lldap	Biblioteca de Python para interactuar con servidores LDAP, utilizada en la aplicación Flask para autenticar usuarios contra OpenLDAP.
Secret (Kubernetes)	Objeto de Kubernetes diseñado para almacenar información sensible (contraseñas, tokens, certificados) de forma separada del código de la aplicación.
Service (Kubernetes)	Abstracción que expone un conjunto de Pods como un servicio de red estable, con IP y nombre DNS fijos independientemente del ciclo de vida de los Pods.
Zero-Trust	Modelo de seguridad que no asume confianza implícita dentro de ninguna red. Todo tráfico, incluso el interno, debe ser explícitamente autorizado.

11. Manual de Despliegue

11.1 Requisitos Previos

Componente	Versión Mínima	Verificación
Docker Desktop / Docker Engine	24.x	<code>docker --version</code>
Minikube	1.32.x	<code>minikube version</code>
kubectl	1.28.x	<code>kubectl version --client</code>
Git	2.40.x	<code>git --version</code>
Python	3.11.x	<code>python3 --version</code>
Node.js (para tools)	18.x	<code>node --version</code>

11.2 Variables de Entorno y Secrets

Las variables sensibles NO deben estar en el código. Se configuran como Kubernetes Secrets. Ejemplo de archivo secrets.yaml (los valores deben ser base64-encoded):

```
# Codificar en base64: echo -n 'mipassword' | base64
```

Secretos requeridos:

Secret Name	Key	Descripción
sql-secret	SA_PASSWORD	Contraseña del usuario sa de SQL Server
sql-secret	DB_NAME	Nombre de la base de datos (inventario)
mongo-secret	MONGO_INITDB_ROOT_USERNAME	Usuario root de MongoDB
mongo-secret	MONGO_INITDB_ROOT_PASSWORD	Contraseña root de MongoDB
ldap-secret	LDAP_ADMIN_PASSWORD	Contraseña del administrador OpenLDAP
app-secret	SECRET_KEY	Clave secreta para sesiones Flask
app-secret	LDAP_BIND_DN	DN de bind para consultas LDAP

11.3 Pasos de Instalación y Despliegue

Paso 1: Iniciar Minikube con Calico como CNI (OBLIGATORIO para NetworkPolicies)

```
minikube start --cni=calico --memory=4096 --cpus=4
```

Paso 2: Verificar que Calico está corriendo

```
kubectl get pods -n kube-system | grep calico
```

Paso 3: Clonar el repositorio del proyecto

```
git clone https://github.com/ITU-EGI/ecosistema-inventario-seguro.git
cd ecosistema-inventario-seguro
```

Paso 4: Crear el namespace del proyecto

```
kubectl apply -f k8s/namespace.yaml
```

Paso 5: Crear los Secrets con credenciales

```
kubectl apply -f k8s/secrets.yaml -n inventario-seguro
```

Paso 6: Desplegar el servidor de identidad OpenLDAP

```
kubectl apply -f k8s/ldap/ -n inventario-seguro
kubectl wait --for=condition=Ready pod -l app=ldap-service -n inventario-seguro --timeout=120s
```

Paso 7: Desplegar SQL Server y aplicar esquema

```
kubectl apply -f k8s/sql-db/ -n inventario-seguro
kubectl wait --for=condition=Ready pod -l app=ubicacion-db -n inventario-seguro --timeout=180s
kubectl exec -n inventario-seguro deploy/ubicacion-db -- /opt/mssql-tools/bin/sqlcmd -S localhost -U sa -P $SA_PASSWORD -i /scripts/create_db.sql
```

Paso 8: Desplegar MongoDB e insertar documentos de hardware

```
kubectl apply -f k8s/mongo-db/ -n inventario-seguro
kubectl wait --for=condition=Ready pod -l app=inventario-db -n inventario-seguro --timeout=120s
kubectl exec -n inventario-seguro deploy/inventario-db -- mongosh -u root -p $MONGO_PASSWORD /scripts/seed.js
```

Paso 9: Desplegar la aplicación web Flask

```
kubectl apply -f k8s/frontend/ -n inventario-seguro
```

Paso 10: Aplicar NetworkPolicies (Zero-Trust)

```
kubectl apply -f k8s/network-policies/ -n inventario-seguro
```

Paso 11: Obtener URL de acceso

```
minikube service inventario-web-svc -n inventario-seguro --url
```

Paso 12 (opcional): Configurar minikube tunnel para LoadBalancer

```
minikube tunnel # Dejar corriendo en terminal separada
```

11.4 Verificación del Despliegue

```
kubectl get all -n inventario-seguro
kubectl get networkpolicies -n inventario-seguro
kubectl logs deploy/inventario-web -n inventario-seguro
```

11.5 Acceso a MongoDB Shell (Queries Manuales)

```
kubectl exec -it deploy/inventario-db -n inventario-seguro -- mongosh
use inventario
db.hardware.find({tipo: 'laptop'}).pretty()
db.hardware.find({'ram.cantidad_gb': {$gte: 16}})
db.hardware.updateOne({id_equipo: 'PC-LAB1-001'}, {$set:
{'ram.cantidad_gb': 32}})
db.hardware.deleteOne({id_equipo: 'PC-LAB2-999'})
```

12. Pruebas Documentadas

12.1 Estrategia de Testing

Dado el contexto académico y las restricciones del entorno de laboratorio, se implementa una estrategia de testing pragmática que combina pruebas manuales funcionales con verificaciones automáticas de infraestructura. Las pruebas se organizan en tres categorías:

- Pruebas funcionales de la aplicación: verifican que las funcionalidades del frontend operan correctamente (login, CRUD, vistas).
- Pruebas de integración: verifican la comunicación entre la aplicación y cada servicio de backend.
- Pruebas de seguridad de red: verifican que las NetworkPolicies aplican correctamente el modelo Zero-Trust.

12.2 Casos de Prueba Funcionales

ID	Prueba	Precondición	Pasos	Resultado Esperado	Estado
PT-01	Login con credenciales válidas	OpenLDAP activo con usuario admin/Admin123	1. Ir a /login. 2. Ingresar admin/Admin123. 3. Submit	Redirige a /equipos con sesión activa	✓ PASS
PT-02	Login con credenciales inválidas	OpenLDAP activo	1. Ir a /login. 2. Ingresar usuario/wrongpassword. 3. Submit	Muestra mensaje de error, permanece en /login	✓ PASS
PT-03	Listado de equipos	Session activa, SQL Server con datos	1. Autenticarse. 2. Ir a /equipos	Tabla con todos los equipos de la DB	✓ PASS
PT-04	Detalle de equipo con hardware	SQL y MongoDB con datos del mismo id_equipo	1. Click en equipo. 2. Ver /equipo/PC-LAB1-001	Vista unificada con sección SQL y sección MongoDB	✓ PASS

PT-05	Detalle sin datos MongoDB	SQL con equipo, MongoDB sin documento para ese ID	1. Click en equipo sin hardware. 2. Ver detalle	Muestra datos SQL, sección hardware indica 'Sin datos'	✓ PASS
PT-06	Crear hardware (Admin)	Sesión con rol Admin	1. Ir a /hardware/nuevo. 2. Completar form. 3. Submit	Documento insertado en MongoDB, redirige a detalle	✓ PASS
PT-07	Cerrar sesión	Sesión activa	1. Click en Logout. 2. Intentar acceder a /equipos	Redirige a /login, cookie de sesión eliminada	✓ PASS

12.3 Pruebas de Seguridad de Red (NetworkPolicies)

ID	Prueba	Comando Utilizado	Resultado Esperado	Estado
PS-01	MongoDB NO accesible desde pod LDAP	kubectl exec ldap-pod -n inventario-seguro -- curl -m 3 inventario-db:27017	Connection refused o timeout	✓ PASS
PS-02	SQL Server NO accesible desde pod MongoDB	kubectl exec mongo-pod -n inventario-seguro -- nc -zv ubicacion-db 1433	Connection refused	✓ PASS
PS-03	Frontend SÍ puede consultar SQL Server	kubectl logs deploy/inventario-web -n inventario-seguro grep 'SQL'	Logs muestran queries exitosas	✓ PASS
PS-04	Frontend SÍ puede consultar MongoDB	kubectl logs deploy/inventario-web -n inventario-seguro grep 'mongo'	Logs muestran consultas exitosas	✓ PASS
PS-05	NetworkPolicies aplicadas en el namespace	kubectl describe networkpolicies -n inventario-seguro	Lista las 6 políticas activas	✓ PASS
PS-06	Acceso externo solo por puerto 30080	curl http://<minikube-ip>:1433 (debe fallar)	Connection refused desde red externa	✓ PASS

12.4 Herramientas de Testing

Herramienta	Uso
kubectl exec	Ejecutar comandos dentro de los pods para verificar conectividad de red

kubectl logs	Inspeccionar logs de los contenedores para verificar queries y errores
kubectl describe networkpolicies	Verificar que las políticas de red están aplicadas correctamente
mongosh (en contenedor)	Ejecutar queries manuales de búsqueda, actualización y eliminación en MongoDB
sqlcmd (en contenedor)	Ejecutar queries SQL para verificar datos en SQL Server
curl / nc	Probar conectividad TCP entre pods para validar NetworkPolicies
Navegador web	Pruebas funcionales del frontend Flask/Jinja2

13. Limitaciones Conocidas

ID	Limitación	Impacto	Mitigación
LIM-01	Simulación de firewall con GFW en lugar de pfSense real	La simulación no replica completamente las capacidades de un firewall de red dedicado	Se documenta explícitamente la simulación y se indican las reglas que implementaría pfSense
LIM-02	OpenLDAP en lugar de Active Directory institucional real	No simula la integración Kerberos ni las GPO de AD real	OpenLDAP implementa el protocolo LDAP estándar; la autenticación es funcionalmente equivalente para el scope del proyecto
LIM-03	Minikube no es apto para entorno de producción	Un único nodo no garantiza alta disponibilidad ni failover real	Definido en alcances; el proyecto es académico y no contempla HA
LIM-04	Sin HTTPS/TLS en el frontend	Las credenciales viajan en texto plano por la red del laboratorio	Se documenta como mejora futura; en producción se requiere certificado TLS y Ingress con terminación SSL
LIM-05	Persistencia de datos dependiente del host Minikube	Si el nodo Minikube se detiene sin PersistentVolume configurado, los datos se pierden	Los manifiestos incluyen PersistentVolumeClaims para SQL Server y MongoDB
LIM-06	Sin mecanismo de backup automatizado	Los datos del inventario no tienen punto de recuperación ante fallo	Mejora futura: CronJob Kubernetes para dumps periódicos
LIM-07	Autenticación Flask basada en sesiones del servidor	Con múltiples réplicas del frontend, las sesiones no son compartidas	Para el scope actual (1 réplica) no hay impacto; solución futura: Redis para sesiones distribuidas

LIM-08	SQL Server Express tiene límite de 10 GB por base de datos	Limitación de la versión Express, no existe en Developer o Enterprise	Para el parque de computadoras del ITU no representa un problema práctico
--------	--	---	---

14. Alcances y Límites del Sistema

14.1 Qué Cubre el Sistema

- Inventario centralizado de computadoras de laboratorios del ITU.
- Gestión completa (CRUD) de datos de ubicación, asignación y mantenimiento en SQL Server.
- Gestión completa (CRUD) de componentes de hardware en MongoDB.
- Autenticación centralizada mediante OpenLDAP con diferenciación de roles.
- Despliegue contenerizado en Minikube con NetworkPolicies Zero-Trust.
- Simulación de parametrización de red mediante reglas GFW.
- Versionado completo del proyecto en repositorio Git con historial de commits por integrante.
- Acceso a MongoDB desde shell del contenedor para queries manuales.
- Correlación automática de datos entre SQL Server y MongoDB por id_equipo.

14.2 Qué NO Cubre el Sistema

- Inventario de dispositivos que no sean computadoras (impresoras, proyectores, routers).
- Módulo de facturación, compras o gestión patrimonial de activos.
- Integración con sistemas académicos del ITU (SIU Guaraní, Moodle, etc.).
- Notificaciones automáticas por correo electrónico o push para fechas de mantenimiento.
- Aplicación móvil nativa (iOS/Android).
- Alta disponibilidad (múltiples réplicas de bases de datos con replicación).
- Comunicación cifrada HTTPS/TLS en el frontend.
- Despliegue en cloud público (AWS, Azure, GCP).
- Módulo de reportes o dashboards analíticos avanzados.
- Integración con herramientas de monitoreo (Prometheus, Grafana).

14.3 Límites Operativos

Parámetro	Límite Definido	Justificación
Usuarios concurrentes	Hasta 30 (capacidad laboratorio ITU)	Diseño orientado a un único laboratorio académico

Equipos inventariados	Hasta 500 equipos	Escala del parque de computadoras del ITU; SQL Express soporta hasta 10 GB
Entorno de despliegue	Minikube en PC de laboratorio	Restricción de la consigna académica
Disponibilidad objetivo	Horario lectivo (no 24/7)	Sistema académico, no crítico de negocio
Retención de logs	No definida (mejora futura)	Fuera del scope del proyecto actual

15. Conclusión

15.1 Resultado Final del Proyecto

El Ecosistema de Inventario Seguro (EIS) representa una solución técnica completa y coherente al problema de gestión de inventario tecnológico del ITU. El proyecto integra exitosamente conocimientos de cinco disciplinas distintas —desarrollo web, bases de datos relacionales, bases de datos NoSQL, administración de sistemas y redes, y seguridad en infraestructura—, demostrando la capacidad del equipo de construir un sistema de complejidad real en un entorno académico controlado.

El ecosistema entrega un frontend Flask funcional con autenticación LDAP, vistas combinadas de datos SQL y MongoDB, dos bases de datos especializadas correctamente diferenciadas por tipo de dato, un servidor de identidad OpenLDAP operativo, políticas de red Zero-Trust verificadas con Calico, y simulación de perimetría con GFW, todo orquestado en Minikube con manifiestos de Kubernetes completos y versionado en Git.

15.2 Beneficios Obtenidos

- Centralización total del inventario: un único punto de acceso para consultar ubicación y hardware de cualquier equipo del laboratorio.
- Seguridad por diseño: el modelo Zero-Trust garantiza que incluso ante el compromiso de un pod, el radio de impacto está limitado por las NetworkPolicies.
- Trazabilidad completa: el historial de commits en Git evidencia la evolución del proyecto y la contribución de cada integrante.
- Flexibilidad del modelo de datos: la elección de MongoDB para hardware permite agregar nuevos tipos de equipos sin modificar el esquema.
- Reproducibilidad del entorno: los manifiestos de Kubernetes garantizan que el ecosistema puede levantarse de forma idéntica en cualquier PC del laboratorio.
- Formación integral del equipo: cada integrante desarrolló competencias técnicas profundas en su área de responsabilidad, con visión del sistema completo.

15.3 Posibles Mejoras Futuras

Mejora	Descripción	Prioridad
TLS/HTTPS	Implementar certificado SSL con Let's Encrypt y Ingress Controller con terminación TLS para cifrar el tráfico del frontend.	Alta
Alta Disponibilidad	Configurar replicación en MongoDB (Replica Set) y Always On para SQL Server, con múltiples réplicas del frontend.	Alta
Sesiones Distribuidas	Integrar Redis como backend de sesiones para soportar múltiples réplicas del pod inventario-web.	Media
Notificaciones de Mantenimiento	Implementar un CronJob de Kubernetes que envíe alertas por correo cuando se acercan fechas de mantenimiento.	Media
Monitoreo con Prometheus/Grafana	Instrumentar la aplicación Flask con métricas Prometheus y visualizarlas en Grafana para monitoreo operacional.	Media
CI/CD Pipeline	Implementar un pipeline de integración y despliegue continuo con GitHub Actions que construya y publique las imágenes Docker automáticamente.	Media
Backup Automatizado	CronJob de Kubernetes que ejecute dumps periódicos de SQL Server y MongoDB hacia almacenamiento persistente externo.	Alta
API REST	Separar el backend en una API REST independiente (Flask-RESTful) para permitir integración con otros sistemas del ITU.	Baja
Módulo de Reportes	Dashboard con gráficas de distribución de equipos por aula, estado de mantenimiento y antigüedad del parque tecnológico.	Baja
Active Directory Real	Reemplazar OpenLDAP por integración con el AD institucional real del ITU para autenticación con credenciales de red del dominio.	Alta

El EIS sienta las bases de una plataforma escalable que, con las mejoras descritas, podría evolucionar de un proyecto académico a una herramienta de gestión real adoptable por el área de infraestructura del ITU. La arquitectura de microservicios, el modelo de seguridad y la separación de responsabilidades entre los motores de base de datos garantizan que dicha evolución sea técnicamente viable sin necesidad de rediseñar el sistema desde cero.